

第17章 多重继承

17.1 多重继承

当一个派生类拥有两个或更多的基类时，这种继承关系被称为多重继承。尽管通常建议用组合来替代私有或保护继承，但仅仅采用公有继承的多重继承也能在设计中提供极大方便。例如，假设已存在教师(Teacher)类和学生(Student)类，我们可以通过以下方式轻松定义一个在职博士生类：

```
1 class Doctor : public Teacher, public Student {
2     // 略
3 };
```

尽管多重继承为程序设计带来了便利，但它也存在一些潜在的问题：

首先，在公有继承中，子类向父类的类型转换总是安全的，无论是指针、引用还是对象形式的转换。然而，在多重继承的情况下，由于派生类对象包含多个基类子对象，其内存布局会变得复杂，具体来说，派生类对象的起始地址仅与第一个基类子对象的地址相同，而不会与其他基类子对象的起始地址一致。因此，为了实现向上转型，编译器可能需要执行额外的内部操作，这会降低转换效率。

其次，多重继承可能导致命名冲突。如果多个基类中存在同名的成员函数或数据成员，派生类中就会出现命名冲突。一般情况下，可以使用 using 关键字来解决此类冲突。例如：

```
1 #include <iostream>
2 class Parent1 {
3 public:
4     void f( )    { std::cout << "Parent1::f( )\n"; }
5     void g1( )   { std::cout << "Parent1::g1( )\n"; }
6     int mNum = 1;
7 };
8 class Parent2 {
9 public:
10    void f( )     { std::cout << "Parent2::f( )\n"; }
11    void g2( )     { std::cout << "Parent1::g2( )\n"; }
12    int mNum = 2;
13 };
14 class Child :public Parent1, public Parent2 {
15 public:
16    void k( )      { std::cout << "Child::k( )\n"; }
17    int mData = 3;
```

```

18 };
19 class ChildWithUsing :public Parent1, public Parent2 {
20 public:
21     using Parent1::f;    //使用 using,解决命名冲突
22     using Parent2::mNum; //使用 using,解决命名冲突
23     void k( ) { }
24     int mData = 3;
25 };
26 int main( ) {
27     Child child;
28     //child.mNum = 101;    //非法
29     child.Parent1::mNum = 101; //合法
30     std::cout << "child.Parent1::mNum=" << child.Parent1::mNum << std::endl; //输出 101
31     std::cout << "child.Parent2::mNum=" << child.Parent2::mNum << std::endl; //输出 2
32     //child.f( );        //非法
33     child.Parent1::f( ); //合法, 输出 Parent1::f( )
34     child.Parent2::f( ); //合法, 输出 Parent2::f( )
35     ChildWithUsing child2; //ChildWithUsing 类中使用了 using 语句
36     child2.mNum = 201;    //合法
37     child2.f( );        //合法, 输出 Parent1::f( )
38     std::cout << "child2.Parent1::mNum=" << child2.Parent1::mNum << std::endl; //输出 1
39     std::cout << "child2.Parent2::mNum=" << child2.Parent2::mNum << std::endl; //输出 201
40 }

```

17.2 命名冲突的本质

在多重继承的情况下, 派生类中可能出现命名冲突, 既包括成员函数的命名冲突, 也包括数据成员的命名冲突。其中, 对于成员函数命名冲突, 由于每个成员函数的代码块总是唯一的, 因此在冲突发生时, 通过限定类名就可以唯一地确定函数入口地址, 也就解决了函数的命名冲突问题; 但对于数据成员的命名冲突, 只限定类名是不能完全解决冲突的。在派生类中每个同名的数据成员会占据独立的存储空间, 如果这些同名数据确实来源于不同的基类, 那么是可以通过限定类名区分的, 如上节例子; 但如果同名数据来源于同一个祖先类, 那么要解决这样的命名冲突是困难的。

从上可见, 多重继承下的命名冲突本质上是数据成员的命名冲突, 特别是数据成员来源于同一个祖先类时, 这时的命名冲突是致命的, 是必须要解决的。例如半圆型结构:

```

1 struct A {
2     int mA;
3 };
4 struct B: A {
5     int mB;

```

```

6   };
7   struct C: A,B {
8       int mC;
9   };
10  int main( ) {
11      C c;
12      //c.mA = 10;非法
13  }

```

这里结构 C 对象的内存布局是非标准布局, 包含两个 mA, 一个 mB 和一个 mC, 而且两个 mA 的根本来源都是结构 A, 要访问结构 C 中的 mA 时, 必须明确区分是哪个 mA。即使在 C 中使用了 using A::mA; 或者 using B::mA; 语句, main() 中的语句 c.mA=10; 也是非法的。这里以 using B::mA; 语句为例, 做一下解释: 假如在 C 中使用了 using B::mA; 语句, 那么表示在 C 中引入了一个新的数据成员, 其外部可见的名字为 mA, 其内部名是 B::mA, 而 B 中的 mA 是从 A 继承来的, 即这个新的数据成员 mA 的内部名是 A::mA; 这样当在 main() 中访问 c.mA 时, 就是访问内部名为 A::mA 的数据, 但 c 中有两个内部名均为 A::mA 的数据, 因此产生编译错误。对于多重继承中的命名冲突, 另一个著名的例子就是钻石结构, 也称菱形结构。

17.3 钻石结构

看下面的代码:

```

1  #include <iostream>
2  struct A {
3      void fA( ) { mA += 10; }
4      int  mA = 1;
5  };
6  struct B : A {
7      void fB( ) { }
8      int  mB = 2;
9  };
10 struct C : A {
11     void fC( ) { }
12     int  mC = 3;
13 };
14 struct D : B, C {
15     using B::fA;
16     using C::mA;
17     void fD( ) {
18         std::cout << "B::mA=" << B::mA << std::endl;

```

```

19         std::cout << "B::mB=" << mB << std::endl;
20         std::cout << "C::mA=" << C::mA << std::endl;
21         std::cout << "C::mC=" << mC << std::endl;
22         std::cout << "D::mD=" << mD << std::endl;
23         std::cout << "-----" << std::endl;
24     }
25     int mD = 4;
26 };
27 int main( ) {
28     D d;
29     d.fD( );    //d 的布局: 1 2 1 3 4
30     //d.mA = 14; //非法
31     //d.fA( );  //非法
32
33     d.A::fA( );
34     d.fD( );    //d 的布局: 11 2 1 3 4
35     d.B::fA( );
36     d.fD( );    //d 的布局: 21 2 1 3 4
37
38     d.C::fA( );
39     d.fD( );    //d 的布局: 21 2 11 3 4
40 }

```

在由 A、B、C 和 D 组成的钻石结构中，虽然在类 D 中使用了两条 using 语句，但在 main 函数中使用语句 d.mA = 14;和 d.fA();仍然是非法的。同时，在 D::fD()中访问 mA 时仍然需要使用类名或结构名进行限定。这表明，using 语句并不能从根本上解决命名冲突问题。此外，需要注意的是，在 main()函数中，尽管 d.A::fA()、d.B::fA()和 d.C::fA()调用的是同一个代码块，并且对象都是 d，但它们的执行结果却不同，这显然不合常理。这也说明，要正确使用结构 D 的对象，使用者必须对 D 的内存布局有清晰的理解。然而，这一要求对于结构 D 的使用者(无论是其他类还是 main 函数)来说显得过于苛刻。

因此，面向对象程序设计语言必须提供有效的机制或方案来解决多重继承中的命名冲突问题。

17.4 C++中的虚基类

在 C++语言中，为解决多重继承下命名冲突的问题，引入了虚基类。例如：

```

1  #include <iostream>
2  struct A {
3      void fA( ) { mA += 10; }
4      int  mA = 1;

```

```

5  };
6  struct B : virtual A { //使用虚基类
7      void fB( ) { }
8      int mB = 2;
9  };
10 struct C : virtual A { //使用虚基类
11     void fC( ) { }
12     int mC = 3;
13 };
14 struct D : B, C {
15     void fD( ) {
16         std::cout << "mA=" << mA << std::endl;
17         std::cout << "mB=" << mB << std::endl;
18         std::cout << "mC=" << mC << std::endl;
19         std::cout << "mD=" << mD << std::endl;
20         std::cout << "-----" << std::endl;
21     }
22     int mD = 4;
23 };
24 int main( ) {
25     D d;
26     d.fD( );    //输出 1 2 3 4
27     d.mA = 14;
28     d.fA( );
29     d.fD( );    //输出 24 2 3 4
30 }

```

在从 A 派生 B 和从 A 派生 C 时，通过在继承列表中使用 `virtual` 关键字引入了虚基类。由于采用了虚基类，在 D 中无需再使用 `using` 语句，即可直接访问 `mA` 数据成员和 `fA` 成员函数。这是因为虚基类改变了 D 的内存布局，确保 `mA` 在 D 中只有一个实例，避免了名称冲突。尽管不同编译器对 D 的构造和内存布局的具体实现有所差异，但总体上其内部布局大致如图 17-1 所示。

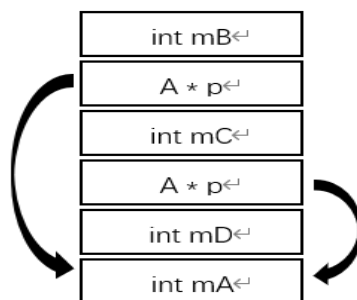


图 17-1 虚基类下派生类的内存布局

D 的构造过程大致如下：首先构造 B，接着构造 C，最后构造成员数据 mD；在构造 B 时，因为基类 A 是虚基类，所以直接构造 B，并设置指针指向稍后才会构造的 A；在构造 C 时，同样由于基类 A 是虚基类，直接构造 C，并设置指针指向稍后构造的 A；然后构造成员数据 mD；最后才构造延迟构造的 A。特别需要注意的是，这个过程中延迟构造的 A 是由 D 负责的，即可以在 D 的成员初始化列表中指定如何构造 A。例如：

```

1  #include <iostream>
2  struct A {
3      A( int n ) : mA( n ) { std::cout << "A(int)\n"; }
4      void fA( ) { }
5      int  mA = 1;
6  };
7  struct B : virtual A { //使用虚基类
8      B( int n ) : A(0),mB( n ) { }
9      void fB( ) { }
10     int mB = 2;
11 };
12 struct C : virtual A { //使用虚基类
13     C( int n ) : A(0),mC( n ) { }
14     void fC( ) { }
15     int mC = 3;
16 };
17 struct D : B, C {
18     //D 的构造函数中，可指明如何构造虚基类 A
19     D( int a, int b, int c, int d ) : B( b ), C( c ), mD( d ), A( a ) { }
20     void fD( ) { std::cout << mA << " " << mB << " " << mC << " " << mD << std::endl; }
21     int mD = 4;
22 };
23 int main( ) {
24     D d(1,2,3,4);
25     d.fD( ); //输出 1 2 3 4
26 }

```

虽然虚基类能够解决多重继承中的命名冲突问题，但它也存在明显的局限性：

1. 真正需要定义的是从 B 和 C 派生出的 D，但在定义 B 和 C 时却使用了虚基类。作为 B 和 C 的定义者，不禁要问：“为什么要使用虚基类呢？”，不使用虚基类，B 和 C 本身也能正常工作。虽然多重继承下的 D 确实需要 B 和 C 使用虚基类，但在定义 B 和 C 时，D 尚未出现。因此，B 和 C 的定义者只需确保 B 和 C 的正确性，而不应为未来的 D 负责。由此可见，要求 B 和 C 使用虚基类显得有些苛刻。

2. 在公有继承中，子类向父类的转换总是安全的。然而，使用虚基类后，内存布局变得更加复杂，这使得在向父类转换时，需要更多的内部处理，从而导致转换效率降低。

总之，为了解决多重继承中的命名冲突问题，C++引入了虚基类，但此方案存局限性，尚未达到完美的程度。在其他面向对象程序设计语言中，针对此问题，通常采用的策略是：通过在语法层面施加更多限制来避免多重继承引发的命名冲突。

17.5 其它的冲突解决方案

17.5.1 单继承

要避免多重继承中的命名冲突问题，最直接的方案就是禁止多重继承，只允许单继承。此方案简单粗暴，虽然避免了多重继承中的问题，但同时也禁止了多重继承的好处。此方案只有个别语言采用，不是主流。

下面的代码就是一种将多重继承改为单继承的实现方式：

```

1  class Parent {
2  public:
3      Parent( int n ) : mNum1( n ) { }
4      void f( );
5  private:
6      int mNum1;
7  };
8  class Other {
9  public:
10     Other( int n ) :mNum2( n ) { }
11     void g( );
12 private:
13     int mNum2;
14 };
15 //多重继承定义子类 Child
16 class Child : public Parent, public Other {
17     Child( int n1, int n2 ) :Parent( n1 ), Other( n2 ) { }
18     void childFunc( );
19 };
20 //用单继承实现同样功能
21 class Child : public Parent {
22 public:
23     Child( int n1, int n2 ) :Parent( n1 ), mOther( n2 ) { }
24     Child( const Child & ) = default; //此例中可省略
25     Child & operator=( const Child & ) = default; //此例中可省略

```

```

26     ~Child( ) = default; //此例中可省略
27 public:
28     void childFunc( );
29     void g( ) { mOther.g( ); } //必须的, 需要保证 Child 的行为与多重继承下一致。
30 public:
31     Other mOther;
32 };

```

在上述示例代码中, 通过单继承实现了与多重继承相同的功能。然而, 这种实现方式存在一些局限性: 首先, Child 类不再继承自 Other 类, 因此 Child 类的对象不能被视为 Other 类的对象; 其次, 相较于直接使用多重继承的简洁高效, 单继承的实现方式更为复杂且难以理解。在例子中, 单继承是通过将 Other 类作为对象成员组合到 Child 类中实现的, 也可以选择使用指针成员(如 Other * mpOther)的方式, 但这会使实现更加复杂, 例如需要为 Child 类自定义构造、析构函数、拷贝构造和赋值函数等。

17.5.2 接口继承

要解决多重继承中的命名冲突, 关键在于防止派生类中出现多个源自同一祖先类的同名数据成员。在设计多重继承的结构时, 如果能够避免这种情况的发生, 就能从根本上解决冲突问题。接口继承是一种有效的方法, 可以满足这一要求, 并且是广泛采用的解决方案之一。

在多重继承中, 如果多个基类中至多只有一个父类具有实例变量, 则本质上不会产生命名冲突。在这种约束下, 数据成员只能以类似于单继承的方式传递, 从而避免了同名数据成员从同一祖先类派生的可能性, 进而解决了命名冲突的问题。特别地, 如果无实例变量的父类均为接口类, 则称这种多重继承为接口继承。

接口类实际上是一种特殊的抽象类, 接口类用于指明其后裔类的公共行为集(也称行为接口), 接口类不能实例化, 但其后裔类可实例化, 接口类中无实例变量, 一般只定义公有行为, 实例方法或类方法均可。有关抽象类和接口类的详细信息, 请参阅虚机制。

需要指出, 接口类中没有实例变量, 也不能实例化, 但没有实例变量的类不都是接口类, 不能实例化的类也不都是接口类。例如:

```

1 //工具类
2 class PackUtility { //PackUtility 无实例变量, 一般不作为接口类看待, 而是工具类
3 public:
4     static void PackBinary( );
5     static void PackText( );
6     static void PackPicture( );

```



```

7     static int  getVer( );
8 private:
9     static int  sVersion;
10 };
11 //抽象类
12 class Shape { //Shape 无实例变量，也不能实例化，但不是接口类，只是普通的抽象类
13 public:
14     virtual ~Shape( ) = default;
15     void show( ) const { cout<<"面积="<< area( ) << endl; }
16     virtual double area( ) const = 0; //纯虚函数
17 };

```

下面给出使用接口继承的示例：

```

1 //顶层的普通类
2 class Device {
3 public:
4     virtual ~Device( ) = default;
5     virtual void sendMsg( ) { }
6 protected:
7     string  mName;
8 };
9 //单继承的普通类
10 class Telephone:public Device {
11 public:
12     void callTo( );
13     void callBy( );
14     virtual void sendMsg( ) override;
15     /* 其它略 */
16 };
17 //接口类
18 class IComputer {
19 public:
20     static const int xx = 1;
21 public:
22     virtual ~IComputer( ) = default;
23     virtual void runApp( ) = 0;
24     virtual void showVideo( ) = 0;
25 };
26 //接口类
27 class IPaper {
28 public:
29     virtual ~IPaper( ) = default;
30     virtual void writeText( ) = 0;

```

```

31  };
32  //接口类派生的接口类
33  class IColorPaper : public IPaper {
34  public:
35      virtual void changeColor( int newColor) = 0;
36  };
37  //接口继承
38  class Mobile : public Telephone, public IComputer, public IColorPaper {
39  public:
40      virtual void sendMsg( ) override;
41      virtual void runApp( ) override;
42      virtual void showVideo( ) override;
43      virtual void writeText( ) override;
44  };

```

对应的 UML 类图如图 17-2 所示:

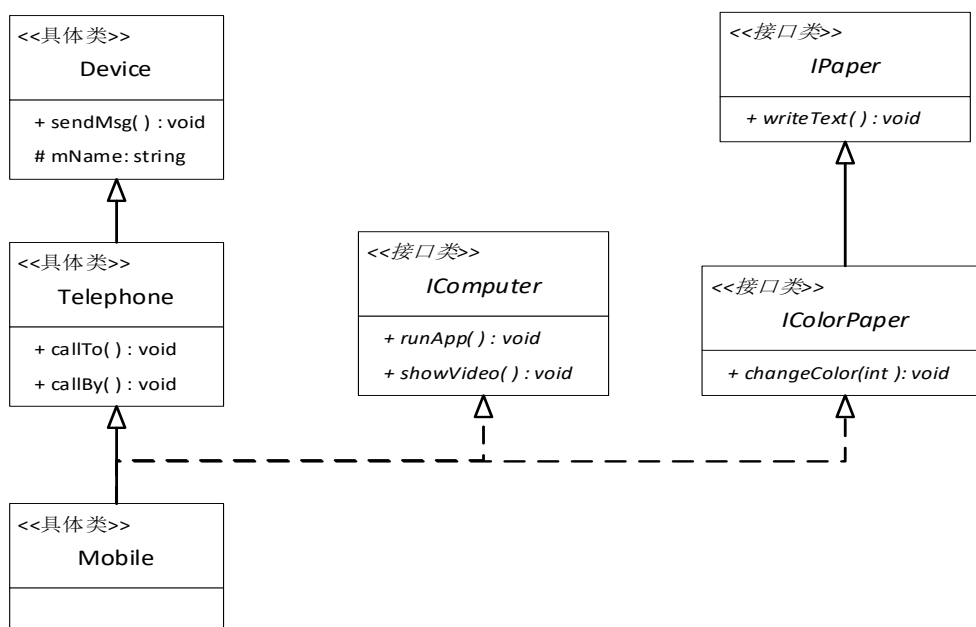


图 17-2 接口继承例

这里 Mobile 类的多个父类中,只有 Telephone 类是含有实例变量的具体类,而其它父类(如 IComputer、IColorPaper)都是接口类,接口类是没有实例变量的。这样 Mobile 类中数据成员的来源就被限制了,要么是 Mobile 类中新定义的,要么是来自具体类 Telephone 类或其祖先类,因此,对于 Mobile 类的数据成员,其本质上相当于单继承了,也就防止了“钻石结构”的产生,从而避免数据成员命名冲突的问题。

17.5.3 混入继承(Mixin)

采用接口继承可以避免命名冲突问题，但父类中最多只能有一个非接口类，这种要求严重限制了多重继承的使用。混入(Mixin)则在接口继承的基础上适当放宽了要求，既能防止命名冲突，又支持多重继承。在混入继承时，允许有多个父类，这些父类既可以是接口类，也可以是非接口类，然而，在所有非接口类的父类中，最多只允许一个父类有祖先类，其他非接口类必须是顶层类。这样可以避免类似钻石结构中的命名冲突问题。从逻辑上，混入继承涵盖了接口继承，接口继承可以视为混入继承的一种特殊情况。例如：

```

1  //日历类
2  class Calender {
3  public:
4      virtual ~Calender( ) { }
5      Date today( ) const;
6      virtual WeekDay weekday(const Date& ) const;
7      /* 略 */
8  };
9  //其它类略
10 //混入(Mixin)继承, Telephone 可以有祖先类, 但 Calender 不能有祖先类
11 class Mobile:public Telephone, public Calender , public IComputer, public IColorPaper {
12 public:
13     virtual void sendMsg( ) override;
14     virtual void runApp( ) override;
15     virtual void showVideo( ) override;
16     virtual void writeText( ) override;
17     virtual WeekDay weekday(const Date& ) const override;
18 };

```

此时的 UML 类图如图 17-3 所示:

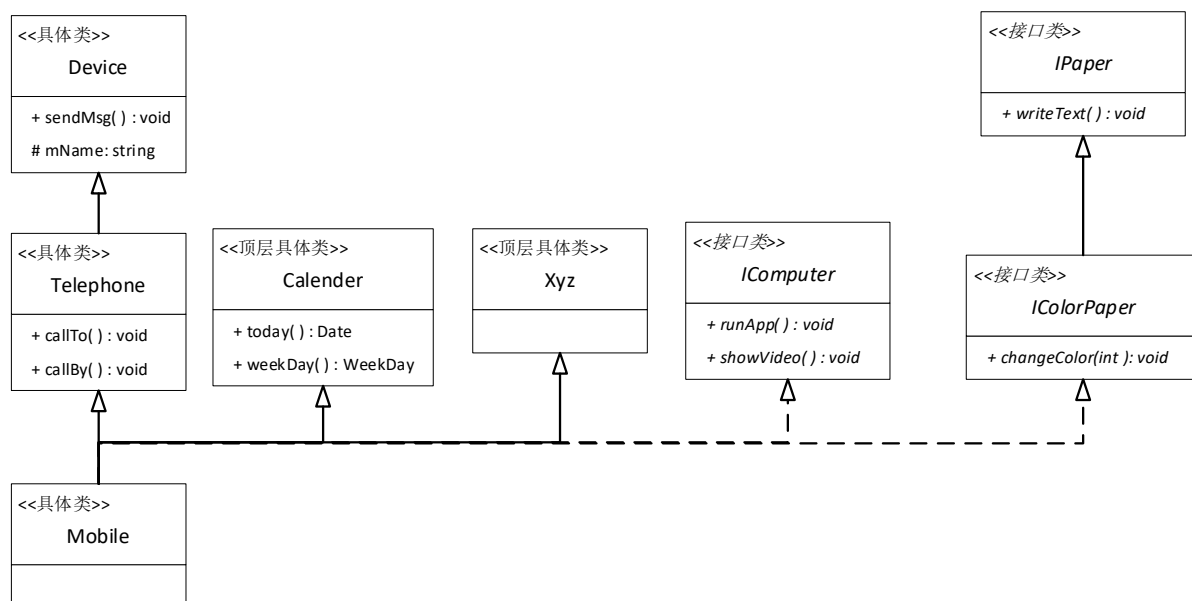


图 17-3 混入继承例

Mobile 类的定义采用了混入继承, 此时 Mobile 的多个父类中可以有多于一个接口类和多个具体类, 但其中的多个具体类中, 至多只有一个(如 Telephone)可以有祖先类, 其它的(如 Calender、Xyz)必须是顶层类。可见, 混入继承放宽了接口继承的限制, 同时避免了 Mobile 的数据成员来自同一个祖先类的可能, 也就变相地防止了“钻石结构”的产生。